



LI.FI

LI.FI Security Review

Catapultar

Security Researcher

Riley Holterhus (rileyholterhus@gmail.com)

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

April 23, 2026

Contents

1	About Researchers	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	3
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	High	4
6.1.1	CATValidator can validate outcomes using unrelated balance increases	4
6.1.2	CatapultarFactory allows _saltPrefix() collisions	4
6.2	Medium	5
6.2.1	CATValidator treats failed balanceOf() reads as zero	5
6.3	Low	6
6.3.1	_validateSignature() does not handle short signatures	6
6.3.2	isValidSignatureNowCalldata() verification path doesn't check publicKeyType	6
6.3.3	ownerOrSelf() and onlyOwnerOrSelf() logic don't check publicKeyType	7
6.4	Informational	7
6.4.1	Cross-chain payloads have no expiry	7
6.4.2	Incorrect comment: type(uint256).max is not disallowed by _useUnorderedNonce()	8
6.4.3	Incorrect NatDoc: entry() documents spend = 0 as balance-of magic value	8
6.4.4	Non-factory deployments expose a window for malicious digest injection	8
6.4.5	Stale P256/WebAuthn key slot not cleared on ownership downgrade	9
6.4.6	helpers/ folder contains unused older contract versions	9
6.4.7	Callers must be careful with fallback calldata interpretation	10
6.4.8	Catapultar storage is not namespaced	10
6.4.9	Key validation logic uses bitwise not() where iszero() would be clearer	10
6.4.10	ERC7821LIFI execution mode comment is outdated	11

1 About Researchers

The team comprised of two distinguished independent security researchers Riley Holterhus and Sujith Somraaj.

Riley Holterhus is an independent security researcher who focuses on Solidity smart contracts. Other than conducting independent security reviews, he works as a Lead Security Researcher at Spearbit, and also searches for vulnerabilities in live codebases. Riley can be reached by email at rileyholterhus@gmail.com, by Telegram at [@holterhus](https://t.me/holterhus) and on Twitter/X at [@rileyholterhus](https://twitter.com/rileyholterhus).

Sujith Somraaj is an independent security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem. He also works as a Lead Security Researcher at Spearbit, advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL. Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- solidity/src/CATValidator.sol
- solidity/src/CallProxy.sol
- solidity/src/Catapultar.sol
- solidity/src/CatapultarFactory.sol
- solidity/src/libs/BitmapNonce.sol
- solidity/src/libs/ERC7821LIFI.sol
- solidity/src/libs/KeyedOwnable.sol
- solidity/src/libs/LibCalls.sol
- solidity/src/libs/LibExecutionConstraint.sol

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 60 hours in total, [LI.FI](#) engaged with two independent researchers Riley and Sujith to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 16 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/catapultar
Commit	b2583cfc
Audit Timeline	March 31, 2026 - April 9, 2026
Methods	Manual Review
Documentation	Medium-High
Test Coverage	Medium-High

Issues Found	
Critical Risk	0
High Risk	2
Medium Risk	1
Low Risk	3
Gas Optimizations	0
Informational	10
Total Issues	16

6 Findings

6.1 High

6.1.1 CATValidator can validate outcomes using unrelated balance increases

Context: [CATValidator.sol](#)

Description: The CATValidator works by recording `balanceOf()` at `outcome.destination` before and after the executor's execution. The balance must increase by at least `outcome.amount`.

There is an issue with this logic if the destination's token balance can increase for a reason unrelated to the solver actually transferring tokens to it.

For example, consider the case where `outcome.destination` is the Catapultar account itself. This is the special case when `outcome.destination == address(0)`, so it seems expected to happen in practice. If the account has an open order to sell an NFT for WETH on OpenSea, the solver could fulfill that order during execution, and the resulting WETH balance increase would be observed as a valid outcome. This is incorrect, since the balance increase came from an unrelated action and the NFT transferred out was never tracked by the CATValidator. Another example is if the account has an unclaimed airdrop: the solver could claim it during execution, and that balance increase would also contribute to the diff check.

This is similar to an issue previously identified in [UniswapX](#).

Recommendation: Consider preventing this issue by either doing a direct `transferFrom()` of the output tokens from the executor, or use a dedicated address for the balance diff logic.

LIFI: Resolved in [PR 23](#).

Auditors: Verified. Tokens now need to be sent to the CATValidator itself, which transfers all tokens to the outcome destination after it verifies its own token balance meets the outcome threshold. One consequence of this design is that duplicate tokens in the `outcomes` array are not supported. After discussion with the team, this was confirmed to be intentional.

6.1.2 CatapultarFactory allows `_saltPrefix()` collisions

Context: [CatapultarFactory.sol#L156-L165](#)

Description: The `_saltPrefix()` function calculates its return value differently depending on the key type: for ECDSA0rSmartContract, it uses the raw owner value directly, while for P256 and WebAuthnP256 keys, it hashes the key type and owner and then takes the first 20 bytes.

```
function _saltPrefix(
    KeyedOwnable.PublicKeyType ktp,
    bytes32[] calldata owner
) internal pure returns (address) {
    if (ktp == KeyedOwnable.PublicKeyType.ECDSA0rSmartContract && owner.length == 1) {
        return address(uint160(uint256(owner[0])));
    } else {
        return address(bytes20(EfficientHashLib.hash(bytes32(uint256(uint8(ktp))),
            ↪ EfficientHashLib.hash(owner))));
    }
}
```

Since the owner is unrestricted user input, an attacker can simply choose an ECDSA0rSmartContract owner value equal to the prefix that would be derived from a victim's intended P256 and WebAuthnP256 key. For example, if a victim wants to deploy a P256 account and hashing produces a salt prefix of `0x1234...abcd`, an attacker can compute that same value off-chain and simply create an ECDSA0rSmartContract account with `owner[0] = 0x1234...abcd`. These two configurations would result in the same `_saltPrefix()` value.

This means that users can deploy the same salt with multiple different types of owners, and only the first deployment would succeed. If an attacker intentionally deploys using a fake ECDSA0rSmartContract owner when the user intended to use a specific P256 or WebAuthnP256 owner, the account will be bricked because the address

has already been taken with an owner that no one controls. Since users may receive funds at their pre-calculated addresses, especially in multichain flows, this could result in frozen funds.

Recommendation: Consider deriving the salt prefix using the same hashing scheme for all key types.

LIFI: Fixed in [PR 20](#).

Auditors: Verified. The `ownerInSalt()` modifier has been removed in favor of a new `_salt()` function. This function hashes the `preSalt` (a user-supplied inner salt), the key type, and the length and contents of the `owner` array to create the salt ultimately used in `create2`. This ensures that each deployment is unique for a given key type and owner array.

6.2 Medium

6.2.1 CATValidator treats failed `balanceOf()` reads as zero

Context: [CATValidator.sol#L129-L135](#)

Description: The CATValidator uses `SafeTransferLib.balanceOf()` to measure the before and after token balances at each `outcome.destination`. Note that `SafeTransferLib.balanceOf()` implementation silently returns 0 when the underlying `balanceOf()` call fails or returns too little data:

```
function balanceOf(address token, address account) internal view returns (uint256 amount) {
    /// @solidity memory-safe-assembly
    assembly {
        mstore(0x14, account)
        mstore(0x00, 0x70a0823100000000000000000000000000000000)
        amount :=
            mul(
                mload(0x20),
                and(
                    gt(returndatasize(), 0x1f),
                    staticcall(gas(), token, 0x10, 0x24, 0x20, 0x20)
                )
            )
    }
}
```

This is dangerous because a failed read is treated as an actual zero balance. If a token can be made to fail the first `balanceOf()` call but succeed the second, then CATValidator may observe an artificial positive difference even though no real increase happened. In other words, a destination with an existing balance could be treated as if it started at zero purely because the first read failed.

One possible mechanism to make this happen would be to find a token whose `balanceOf()` behavior depends on external state, which can be altered during the executor's control flow. Another theoretical attack would be to make the first `balanceOf()` fail with out-of-gas and let the later read succeed, perhaps relying on the 63/64 rule from EIP150, although that path seems less realistic.

No concrete example of this was found from a brief amount of research, but the behavior seems dangerous enough that it should be addressed.

Recommendation: Treat a failed `balanceOf()` as an error rather than silently converting it to zero. To achieve this, consider changing from using the `SafeTransferLib.balanceOf()` implementation to one that reverts on a `balanceOf()` failure.

LIFI: Fixed in [lifinance/catapultar#21](#)

Auditors: Verified fix.

6.3 Low

6.3.1 `_validateSignature()` does not handle short signatures

Context: [KeyedOwnable.sol#L249-L258](#)

Description: In the normal usage of `_validateSignature()`, `signature.length` can be less than 32, and it can even be 0, so both `n - 1` and `n - 32` can underflow in the unchecked block inside `_validateSignature()`.

```
unchecked {
    uint256 n = signature.length;
    // Remove the last byte from the signature
    signature = LibBytes.truncatedCalldata(signature, n - 1);
    // Do the prehash if last byte is non-zero. Select the last word of the signature, then select the
    ↪ last byte
    // of the word.
    if ((uint256(LibBytes.loadCalldata(signature, n - 32)) & 0xff) != 0) {
        digest = EfficientHashLib.sha2(digest); // `sha256(abi.encode(digest))`.
    }
}
```

This can lead to unexpected behavior. This does not appear exploitable as it would simply fail downstream decoding and verification.

Note that zero-length signatures are something that may be desirable to support. For example, if the owner of one Catapultar account is another Catapultar account, then zero-length signatures would be useful since Catapultar supports them via the `approvedDigest` logic. Right now, this would not work correctly because `loadCalldata(signature, n - 32)` would read unrelated surrounding calldata, which can incorrectly cause an extra hash to be performed or skipped.

Recommendation: Consider explicitly handling signatures shorter than 32 bytes before entering the unchecked logic.

LIFI: Addressed in [PR 22](#).

Auditors: Verified. If the signature is empty, the new logic explicitly doesn't shorten the signature length and does not enter the extra hashing path. If the signature is shorter than 32 bytes, the extra hashing path still works correctly. Although `calldataload` reads a full 32-byte word, the assembly masks out everything except the final byte that actually lies within the signature. As a result, bytes outside the signature range can no longer affect control flow.

6.3.2 `isValidSignatureNowCalldata()` verification path doesn't check `publicKeyType`

Context: [KeyedOwnable.sol#L244-L247](#)

Description: The 64/65-byte signature path routes directly into `SignatureCheckerLib.isValidSignatureNowCalldata()` using `_getPublicKeySlice(0)` as an EVM address:

```
if (LibBit.or(signature.length == 64, signature.length == 65)) {
    address account = _asAddressNotDirty(_getPublicKeySlice(0));
    return SignatureCheckerLib.isValidSignatureNowCalldata(account, digest, signature);
}
```

This implicitly assumes that `_getPublicKeySlice(0)` represents an EVM address, which is only true when `publicKeyType == ECDSA0rSmartContract`. For P256 and WebAuthnP256, that value is instead the x-coordinate of the public key.

In practice, this is likely only a theoretical concern, since `_asAddressNotDirty()` requires the upper 12 bytes to be zero, which a random P256 x-coordinate is very unlikely to satisfy. And even if it does, it would still be very unlikely that interpreting that x-coordinate as an EVM address could produce a valid ECDSA or ERC1271 signature. However, a user could intentionally configure a non-ECDSA key with an x-coordinate equal to an address they control, which could result in this path unexpectedly succeeding.

It's also worth noting that P256 signatures will naturally be 64 bytes long and potentially collide with this logic, however this is known behavior and documented:

```
/**
 * ...
 * Notice! P256 signatures are 64 bytes long. This contract has been optimised for ECDSA owners.
 * P256 signatures needs to append 2 bytes to their 64 bytes to get the signature to size 66. The last
 * ↪ byte is a
 * flag for rehashing the digest with SHA256.
 * ...
 */
```

Recommendation: Consider adding `publicKeyType == PublicKeyType.ECDSAOrSmartContract` as a condition to enter the `SignatureCheckerLib.isValidSignatureNowCalldata()` path, so that it can't succeed with P256 owners in edge cases.

LIFI: This is intentional as specified in the code. The code has been optimised for its most common use case.

Auditors: Acknowledged.

6.3.3 `ownerOrSelf()` and `onlyOwnerOrSelf()` logic don't check `publicKeyType`

Context: [KeyedOwnable.sol#L97-L128](#)

Description: The `ownerOrSelf()` and `onlyOwnerOrSelf()` logic checks the `caller()` against the `_OWNER_SLOT` without verifying that `publicKeyType == ECDSAOrSmartContract`.

```
v := eq(caller(), address())
if iszero(v) {
    // If _OWNER_SLOT has higher bits set (not PublicKeyType.ECDSAOrSmartContract) then this will never
    ↪ be
    // true.
    v := eq(caller(), sload(_OWNER_SLOT))
}
```

The assumption is that if the `publicKeyType` is not `ECDSAOrSmartContract`, then the `caller()` will never match `sload(_OWNER_SLOT)`, since this would be the x-coordinate of a P256 key. While this is true in most circumstances, it's technically possible for a user to set up an owner that violates this assumption. For example, a user could set up a P256 key where x is their EVM address and y is an arbitrary bytes32 value, and they would be able to call `onlyOwner()` functions despite the account having a P256 key. This would be unexpected.

This does not appear exploitable since the owner would have to deliberately set this up. However, it'd be safer if it were prevented.

Recommendation: Consider explicitly checking `publicKeyType == PublicKeyType.ECDSAOrSmartContract` in `ownerOrSelf()` and `onlyOwnerOrSelf()` to remove this edge case. Also consider making the same change in the view function `owner()`.

LIFI: Ack. This is not a concern since we this would be self inflicted and the chance of this randomly happening to any EVM address is null.

Auditors: Acknowledged.

6.4 Informational

6.4.1 Cross-chain payloads have no expiry

Context: [LibCalls.sol#L7-L11](#)

Description: The `Calls` struct has (uint256 nonce, bytes32 mode, Call[] calls) with no deadline field. Multichain payloads use `_hashTypedDataSansChainId()`, so the same signature is valid on every chain where the account exists.

A signed payload remains executable indefinitely until the nonce is consumed or manually invalidated. To revoke, the owner must call `invalidateUnorderedNonces()` on every chain separately which is error-prone and gas-intensive.

Recommendation: Add an optional `uint256` deadline to the `Calls` struct (0 = no expiry). Check `block.timestamp <= deadline` in `_validateOpData()` when non-zero to support single signature multichain execution without compromising on long-term security.

LIFI: Acknowledged. Cross-chain payloads are specifically used when we need to provide some setup constraint in the future for a specific account. This could be the case where a user has an account on multiple chains but wants to rotate owner. In these cases, having an expiry is detrimental since it causes these perpetual ownership transfers (which may be needed if the user transfers to the contact on an undeployed chain).

Auditors: Acknowledged.

6.4.2 Incorrect comment: `type(uint256).max` is not disallowed by `_useUnorderedNonce()`

Context: [Catapultar.sol#L228](#)

Description: The comment in `Catapultar._validateOpData()` states:

```
// 0 and type(uint256).max are disallowed by _useUnorderedNonce.
```

Only 0 is explicitly disallowed (checked at `BitmapNonce.sol:33`). `type(uint256).max` is a valid nonce it maps to `wordPos = uint248(type(uint256).max >> 8)` and `bitPos = 255`, which is a usable bitmap position.

Recommendation: Update the comment to:

```
// 0 is disallowed by _useUnorderedNonce.
```

LIFI: Fixed in [lifinance/catapultar#25](#)

Auditors: Verified fix.

6.4.3 Incorrect `NatDoc`: `entry()` documents `spend = 0` as balance-of magic value

Context: [CATValidator.sol#L64](#)

Description: The `NatDoc` for `CATValidator.entry()` states:

```
If a spend is 0 the current balance of the signer will be used.
```

This is incorrect. The actual magic value that triggers the balance-of behavior is `1 < 255` (defined as `SPEND_BALANCE_OF_MAGIC` at `CATValidator.sol:42`), not 0. A spend of 0 simply transfers zero tokens.

An integrator reading the `NatDoc` could set `spend = 0` expecting the full balance to be transferred, resulting in a no-op transfer that silently does nothing.

Recommendation: Update the `NatDoc` to:

```
If a spend is 1 << 255 (SPEND_BALANCE_OF_MAGIC), the current balance of the signer will be used.
```

LIFI: Fixed in [lifinance/catapultar#24](#)

Auditors: Verified fix.

6.4.4 Non-factory deployments expose a window for malicious digest injection

Context: [Catapultar.sol#L115](#)

Description: The `setSignature()` function allows any caller to set approved digests before `init()` is called:

```
function setSignature(bytes32 hash, DigestApproval flag) public payable {
    if (!ownerOrSelf() && _getInitializedVersion() != 0) revert Unauthorized();
```

When `_getInitializedVersion() == 0` (pre-initialization), the authorization check is bypassed. This is intentional as the factory's `deployWithDigest` relies on it to embed an approved call before initializing the account.

The factory flow is safe because `deploy` + `setSignature` + `init` execute atomically in a single transaction. However, if a Catapultar proxy is deployed outside the factory (e.g., via `LibClone` directly) and `init` is called in a separate transaction, an attacker can front-run `init` and inject a malicious approved digest such as `transferOwnership(attacker)`. After the owner initializes, the attacker executes the pre-approved batch without any signature and takes ownership.

Recommendation: Add explicit documentation in the Catapultar contract `NatDoc` and the `README` warning that proxy deployment and initialization must be atomic. For example, in `setSignature()`:

```
/// @notice Allow setting a signed object as validated through owner or self call.
/// @dev WARNING: Before initialization, this function is callable by anyone. Proxy
/// deployment and init() MUST be executed atomically in a single transaction (as the
/// CatapultarFactory does). Deploying a proxy and calling init() in separate transactions
/// exposes a window where an attacker can inject malicious approved digests.
```

LIFI: Acknowledged. Catapultar is not intended to ever be used without a factory.

Auditors: Acknowledged.

6.4.5 Stale P256/WebAuthn key slot not cleared on ownership downgrade

Context: [KeyedOwnable.sol#L187](#)

Description: In `KeyedOwnable._transferOwnership()`, the function only writes `nextKey.length` storage slots when setting a new owner. When transferring from a P256 or WebAuthn key (2 slots: x and y coordinates) to an ECDSA key (1 slot: address), only slot 0 is overwritten. Slot 1 retains the previous key's y-coordinate.

```
function _transferOwnership(PublicKeyType ktp, bytes32[] calldata nextKey) internal {
    if (!_isValidKey(ktp, nextKey)) revert InvalidKey();
    uint256 nextKeyLength = nextKey.length;
    for (uint256 i; i < nextKeyLength; ++i) {
        _setPublicKeySlice(i, nextKey[i]);
    }
    publicKeyType = ktp;
}
```

The same applies to `transferOwnership(address)` which only writes slot 0.

No current code path reads slot 1 when `publicKeyType == ECDSA0rSmartContract`, so this is not exploitable. All access is gated by `_keyTypeLength()` which returns 1 for ECDSA.

Recommendation: Zero out remaining slots on downgrade.

LIFI: Fixed in [lifinance/catapultar#28](#)

Auditors: Verified fix.

6.4.6 helpers/ folder contains unused older contract versions

Context: N/A **Description:** The `helpers/` directory contains older versions of the `CATValidator`, `CallProxy`, and `LibExecutionConstraint` that are no longer relevant. These can be removed.

Recommendation: Delete the `helpers/` directory.

LIFI: Addressed in [PR 17](#).

Auditors: Verified.

6.4.7 Callers must be careful with fallback calldata interpretation

Context: [Catapultar.sol#L261-L264](#)

Description: The fallback can reinterpret calldata in two ways. The `searchAndCall()` function scans the calldata for an embedded `execute()` payload and delegatecalls if found. The `cdFallback()` function treats the calldata as compressed input, decompresses it, and then dispatches the resulting delegatecall. This behavior is intentional, but it means callers need to be careful not to send calldata that accidentally contains a valid embedded `execute()` payload or compressed call for the account when that was not their intent. This is especially relevant for the owner of the account, since they have special privileges when calling `execute()`.

Recommendation: Consider documenting this behavior more explicitly so integrators understand that the fallback may reinterpret calldata as an executable call, and that callers need to be careful not to send calldata that could be interpreted in an unintended way.

LIFI: Ack. This is a rare behaviour and will only happen if the function selector is exactly `not(execute.selector)`. Additionally, the only callers that could cause this to be at risk is the owner the contact itself (as otherwise it would require a validate signature). For a user, calling their own contract with seemingly random calldata is bad practise for any account.

Auditors: Acknowledged.

6.4.8 Catapultar storage is not namespaced

Context: [Catapultar.sol#L55-L71](#)

Description: Some of the storage used by Catapultar and its inherited contracts are using the default Solidity storage slot layout rather than namespaced storage. In particular, `approvedDigest`, `nonceBitmap`, and `publicKeyType` will all occupy regular sequential storage slots in the Catapultar account.

This code will sometimes be used behind an upgradeable proxy. Sequential storage slots can make future upgrades harder to reason about compared to a fully namespaced storage layout.

Recommendation: Consider changing all storage variables to be namespaced storage, to make future upgrades easier to manage and reason about.

LIFI: Ack. We do not believe that using namespaced storage will be an overall security upgrade. The additional complexity caused by manually managing namespace will make accidents more prone to happen.

The intended upgrade path is for replacing the signature scheme used for the account and those slots are manually defined.

Auditors: Acknowledged.

6.4.9 Key validation logic uses bitwise `not()` where `iszero()` would be clearer

Context: [KeyedOwnable.sol#L130-L180](#)

Description: The `_isValidKey()` helper uses `not()` in several places where `iszero()` would be more natural for boolean-style checks. For example:

```
valid := and(valid, not(or(eq(calldataload(key.offset), 0), eq(calldataload(add(key.offset, 0x20)),
→ 0))))
```

`not()` is the bitwise negation of all bits, whereas `iszero()` maps directly to 0 or 1. Using `iszero()` here would make the intent clearer and easier to reason about.

In practice, this does not appear to cause any weird behavior in the current code. Each `not()` is combined with `and(valid, ...)`, and `valid` always starts off as 0 or 1 from `valid := eq(expectedKeyLength, key.length)`, so only the low bit ends up mattering.

Recommendation: Consider replacing these uses of `not()` with `iszero()` to make the boolean logic more explicit.

LIFI: Addressed in [PR 26](#).

Auditors: Verified.

6.4.10 ERC7821LIFI execution mode comment is outdated

Context: [ERC7821LIFI.sol#L62](#)

Description: The bytes layout comment in `_executionModeId()` says that bytes `[2..5]` are reserved for future ERC7579 standardization:

```
// [2..5] ( 4 bytes) Reserved by ERC7579 for future standardization.
```

Note that byte `[2]` is now used for the `isMultichain` logic in the Catapultar code. This may be worth mentioning in the comments.

Recommendation: Consider updating the comment to reflect the usage of byte `[2]`.

LIFI: Updated comment in [PR 27](#).

Auditors: Verified.